Machin learning, 8em5. IHFT.

Exam Date 14/11/2025

Q1.

A. → Role of optimization in ML with Loss function:

A ML learning model has parameters like weights, biases. We can define a loss function $L(y, \hat{y})$ which measures how bad the predictions $\hat{y}$ are compared to true labels $y$.

Optimization is the process of finding parameter values that minimize the loss function on training data. Most commonly, we use gradient-based methods that relatively updates parameters in the direction of negative gradient of loss.

$$\theta \leftarrow \theta - \eta \frac{\partial L}{\partial \theta}$$

where $\theta$ are parameters & $\eta$ is learning rate.

→ Significance of the loss function

The loss function encodes the learning objectives like Regression & classification. The Regression penalize large errors in predicted values & in classification, penalizes wrong class predictions.

Loss function affects Convergance behaviour ie, Smoothness, Robustness to outliers, caliberation of predicted probabilities & bias: variance trade-off when combined with regularization.

→ Different types of loss functions & their impacts.

1. Regression loss.
   - MSE
   - MAE

Mean Squared Errors - strongly penalizes large

(square terms). These are smooth & differentiable which is good for gradient methods. These are sensitive for outliers.

$$MSE = \frac{1}{n}\sum_{i=1}^{n}(y_i - \hat{y}_i)^2$$

Mean Absolute Error (MAE)

$$MAE = \frac{1}{n}\sum_{i=1}^{n}|y_i - \hat{y}_i|$$

It penalizes errors linearly → more robust to outliers. It may converge slower, gradient is constant except at 0.

2. Classification Loss.

• 0-1 Loss

$$L = \begin{cases} 0 & \text{if } y = \hat{y} \\ 1 & \text{otherwise.} \end{cases}$$

• Directly measures misclassification
• Not differentiable.

that is, It is not used for training only evaluation.

• Cross Entropy Loss (Binary)

- For label $y \in \{0, 1\}$ & predicted probability $\hat{p}$: $L = -[y \log(\hat{p}) + (1-y)\log(1-\hat{p})]$
- It encourages correct & confident predictions.
- Convex for logistic regression → guaranteed global min.

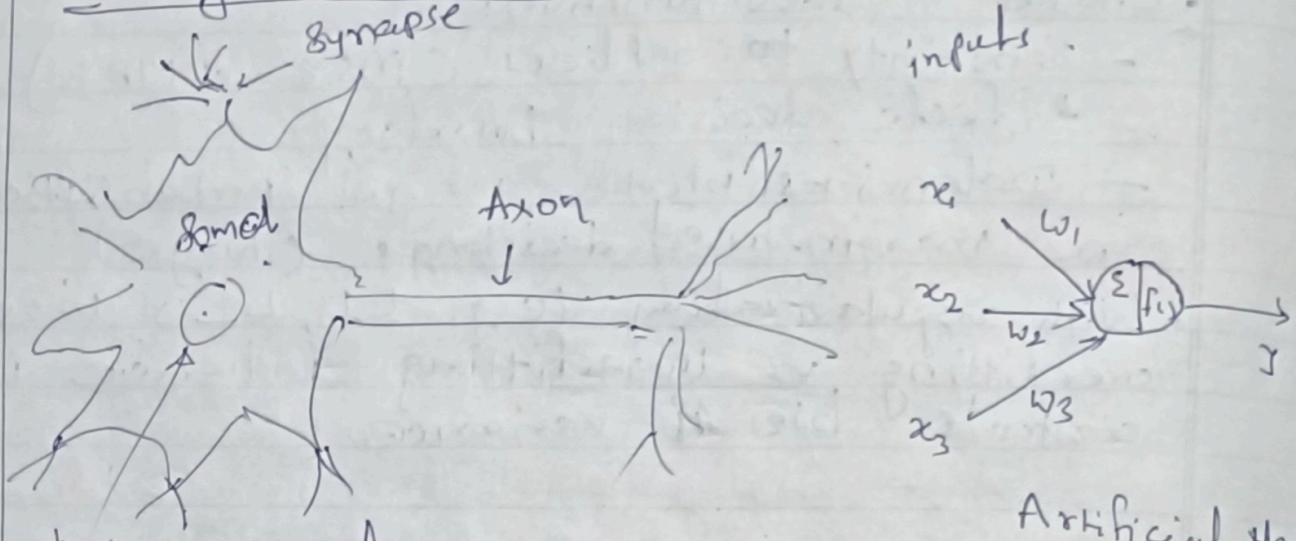
• Categorical Cross Entropy (multi-class)

- For one-hot label vector $y$ & predicted probabilities $\hat{p}$ $L = -\sum_k y_k (\log(\hat{p}_k))$
- Penalizes low probabilities assigned to correct class
- Standard for deep classification networks.

→ Impact of loss function:

- Choice of loss function:
  - Sensitivity to outliers (MSE vs MAE)
  - Affects decision boundary
  - Determines whether we get probabilistic o/p or margin-based decisions (hinge)
- With regularization (e.g. L1, L2) loss controls overfitting vs underfitting & hence it also controls bias & variance.

Q. 1B. Biological Neuron.




Biological Neuron has following main componenles.

- Dendrides: receives i/p signals from other neurons
- Cell body (Soma): processes i/p's.
- Axon: transmits o/p signal
- Synapses: junction where chemical/electrical signal pass to other neurons.
- Neuron fires (generates action potential) if the combined input exceeds a certain threshold.

Artificial Neuron:

An Artificial Neuron (perceptron) is an abstract mathematical model inspired by biological neuron

- Inputs $x_1, x_2, x_3$ are analogous to signal on dendr
- Weights $w_1, w_2, \ldots w_n$ analogous to synaptic strength
- Activation fn $f(\cdot)$ analogous to

. Weighted sum of inputs.

$$z = \sum_{i=1}^{n} w_i x_i + b$$

. connection between neurons across layer mimic biological networks

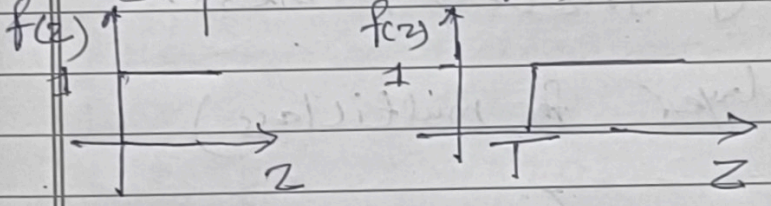
Thus the biological neuron motivates - weighted combination of inputs, Non-linear activation fn and n/w of interconnected units.

• Activation function in Artificial Neurons

- It introduces non-linearity, without it multiple layer collapse to a linear function
- It controls op range (0, 1) or (-1, +1)
- It helps network to learn complex decision boundaries.

Types of Activation functions

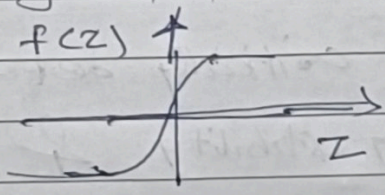
• Step function f(z)




$f(z) = 1$ if $z \geq 0$,

$= 0$ else.

. Sigmoidal funct..




$$f(z) = \frac{1}{1+e^{-z}}.$$

For sigmoid (logistic) function output is (0, 1)
Smooth & differentiable.
Problem: vanishing gradient for large |z|,
slow convergence.

- Hyperbolic tanh

$$f(z) = \tanh(z) = \frac{e^{z} - e^{-z}}{e^{z} + e^{-z}} \quad \text{output } (1, -1)$$

Zero centered, advantage over sigmoid.
Still suffers from vanishing gradient for large |z|.

- ReLU (Rectified Linear Unit)

$$f(z) = \max(0, z)$$

- It is simple & computationally efficient.
- Sparse activations (many zeros)
- Reduces vanishing gradient problem.

- Softmax output layer (for multiclass)
  - for class k

$$f_k(z) = \frac{e^{z_k}}{\sum_j e^{z_j}}$$

- converts raw scores into probability distribution over classes.

Thus activation functions critically determines learning capability, training stability & final accuracy.

Q.2A. DBSCAN - pairwise distances & clustering.

Given

P1 = (2.0, 3.0)
P2 = (1.0, 2.5)
P3 = (4.5, 6.0)
P4 = (5.0, 7.0)
P5 = (8.0, 8.5)
P6 = (10.0, 9.0)

Parameters
eps = 3.0
minPts = 2.0

Pairwise Euclidian distces.

Distance formula for A(x1, y1) & B(x2, y2):

$$d(A,B) = \sqrt{(x_1 - x_2)^2 + (y_1 - y_2)^2}$$

Computing dist for all pairs

d(P1, P2) = $\sqrt{(2-1)^2 + (3-2.5)^2}$ = 1.118
d(P1, P3) = $\sqrt{(2-4.5)^2 + (3-6)^2}$ = 3.905
d(P1, P4) = 5.000
d(P1, P5) = 8.139
d(P1, P6) = 10.000
d(P2, P3) = 4.950
d(P2, P4) = 6.021
d(P2, P5) = 9.220
d(P2, P6) = 11.102
d(P3, P4) = $\sqrt{(4.5-5.0)^2 + (6.0-7.0)^2}$ = 1.118
d(P3, P5) = 4.301
d(P3, P6) = 6.265
d(P4, P5) = 3.354
d(P4, P6) = 5.385
d(P5, P6) = $\sqrt{(8-10)^2 + (8.5-9)^2}$ = 2.062

DBSCAN clustering with eps = 3.0 & minPts = 2

- $\epsilon$ neighbours of a point P: all pts within dist $\leq \epsilon$
- Core points: has at least min points (including itself) in its $\epsilon$ neighbourhood.
- Border points: Itself not core but reachable from a core point.
- Noise: Not part of any cluster.

Step 1: find $\epsilon$-neighbours (dist $\leq 3$)

from the distances:

For P1: Only P2 is within 3 → neighbours: {P1, P2}
P2: Only P1 is within 3 → neighbours: {P2, P1}
P3: Only P4 is within 3 → neighbours: {P3, P4}
P4: only P3 is within 3 → neighbours: {P4, P3}
P5: only P6 is within 3 → neighbours: {P5, P6}
P6: only P5 is within 3 → neighbours: {P6, P5}

Each point has 2 points in its $\epsilon$-neighbourhood (itself + one neighbour).

Since minPts = 2, every point is a core point

Step 2 form clusters.

Cluster 1, $C_1 = \{P_1, P_2\}$, Cluster 2, $C_2 = (P_3, P_4)$

Cluster 3, $C_3 = \{P_5, P_6\}$.

DBSCAN clustering with eps = 3.0 & minPts = 2

- $\epsilon$ neighbours of a point P: all pts within dist $\leq \epsilon$
- Core points: has at least min points (including itself) in its $\epsilon$ neighbourhood.
- Border points: Itself not core but reachable from a core point.
- Noise: Not part of any cluster.

Step 1: find $\epsilon$-neighbours (dist $\leq 3$)

from the distances:

For P1: Only P2 is within 3 → neighbours: {P1, P2}
P2: Only P1 is within 3 → neighbours: {P2, P1}
P3: only P4 is within 3 → neighbours: {P3, P4}
P4: only P5 is within 3 → neighbours: {P4, P3}
P5: only P6 is within 3 → neighbours: {P5, P6}
P6: only P5 is within 3 → neighbours: {P6, P5}

Each point has 2 points in its $\epsilon$-neighbourhood (itself + one neighbour).

Since minPts = 2, every point is a core point

Step 2 Form clusters

Cluster 1; $C_1 = \{P_1, P_2\}$, cluster 2; $C_2 = (P_3, P_4)$

Cluster 3, $C_3 = \{P_5, P_6\}$.

Q 2B. Hyperplane & svm margin maximization.

Hyperplane in svm:

In an n-dimensional feature space, a hyperplane is a flat, (n-1) dimensional subspace, defined as:

$$w^T x + b = 0$$

where

- w is weight vector normal to the hyperplane
- b is bias (offset)
- x is feature vector

In binary classification.

$y \in \{-1, +1\}$, svm uses a hyperplane to separate two classes.

- Points to which $w^T x + b > 0$ are assigned +1
- Points to which $w^T x + b < 0$ are assigned -1.

Idea of maximizing margin.

Margin - dist between the hyperplane & the closest data points (support vectors) from each class.

Svm aims to find the hyperplane that maximizes the margin, which improves generalization.

Geometrically

- Dist of a point x from hyperplane $w^T x + b = 0$

$$dist(x, H) = \frac{|w^T x + b|}{||w||}$$

Svm contains data such that $y_i (w^T x_i + b) \geq 1 \quad \forall i$

- Under these constraints, margin = $\frac{2}{\|w\|}$

So, maximizing margin = minimizing $\|w\|$

Primal optimization problem: (Hard margin)

Given linearly separable data $\{(x_i, y_i)\}$ with $y_i \in \{-1, +1\}$,

$$\min \frac{1}{2}\|w\|^2$$

subject to $y_i(w^T x_i + b) \geq 1 \quad , i = 1, 2, .., N,$

Constraints ensures that correct classification with a margin of at least 1.

Soft Margin (Non-separable case)

Lets introduce slack variable $\xi_i \geq 0$ to allow mis-classification: $\min \frac{1}{2}\|w\|^2 + C \sum_{i=1}^{N} \xi_i$

subject to $y_i(w^T x_i + b) \geq 1 - \xi_i \quad \xi_i \geq 0$

- C controls tradeoff between maximizing margin (small $\|w\|$) and minimizing misclassification error ($\xi_i$)

By considering this sum ignores points far from ~~boundary~~ borderline boundary & focuses on borderline (support) vectors. Maximizing margin reduces overfitting, leading to better generalization. The hyperplane is chosen such that the "worst-classified correct point" (closest to border) is as far as possible.

Q.3A Model complexity, Bias, and variance

Bias: Error is due to overly simplistic assumptions in the model
High bias → model underfits.

Variance: Error due to too much sensitivity to training data.
High variance → model overfits.

- Effect of model complexity.

① Low complexity model
(e.g. linear model for non-linear data)
It cannot capture underlying pattern - underfitting
- High bias low variance.
- Training error: high
- Test error: high
- Model predictions don't change with different training sets.

② High complexity model (e.g. very deep tree / high-degree polynomial)
- Fits training data very closely, including noise results in overfitting
- Low bias high variance.
- Training error: very low (almost zero).
- Test error, high
- predictions change a lot with different training sets.

③ Optimal complexity: Trade-off - choose model complexity such that Training error is reasonably low (bias not too high), Test error minimal - variance under control. Optimal complexity often achieved using: cross validation, Regularization & early stopping.

Q3B. Regression Metrics: MSE, MAE, RMSE, $R^2$

| Actual values | Predicted values |
|---|---|
| $y$ | $\hat{y}$ |
| 4 | 3.5 |
| -0.5 | 1.0 |
| 5 | 7 |
| 7 | 8 |

$e_i = y_i - \hat{y}_i$

$e_1 = 0.5$

$e_2 = -1.5$

$e_3 = -2$

$e_4 = -1$

① Mean Square Error (MSE): $MSE = \frac{1}{n}\sum_{i=1} e_i^2$

$\therefore MSE = \frac{1}{4}(e_1^2 + e_2^2 + e_3^2 + e_4^2) = \frac{1}{4}(0.5^2 + (-1.5)^2 + (-2)^2 + (-1)^2$

$MSE = \frac{7.5}{4} = 1.875$

② Mean absolute error (MAE): $MAE = \frac{1}{n}\sum_{i=1}^{n} |e_i|$

$\therefore MAE = \frac{1}{4}(|e_1| + |e_2| + |e_3| + |e_4|) = \frac{1}{4}(|0.5| + |-1.5| + |-2| + |-1|$

$MAE = \frac{5}{4} = 1.25$

③ Root Mean Squared Error (RMSE)

$RMSE = \sqrt{MSE} = \sqrt{1.875} = 1.369$

④ R-squared ($R^2$)

$R^2 = 1 - \frac{SS_{Res}}{SS_{tot}}$

$SS_{Res} = \sum e_i^2 = 7.5$

Mean of Actual values $= \frac{4 + (-0.5) + 5 + 7}{4} = 3.875$

$SS_{tot} = \sum (y_i - \bar{y})^2$

$\therefore 4 - 3.875 = 0.0125 \Rightarrow (0.0125)^2 = 0.000156$

$-0.5 - 0.3875 = -4.375 \Rightarrow (-4.375)^2 = 19.140625$

$5 - 0.3875 = 1.125 \Rightarrow (1.125)^2 = 1.265625$

$7 - 0.3875 = 3.125 \Rightarrow (3.125)^2 = 9.765625$

$\therefore SS_{Tot} = 0.015625 + 19.140625 + 1.265625 + 9.765625$

$= 30.1875$

Now $R^2 = 1 - \frac{SS_{Res}}{SS_{Tot}} = 1 - \frac{7.5}{30.1875}$

$= 1 - 0.24845$

$= 0.752$

$\boxed{R^2 = 0.752}$

Q.3C Type of ML for streaming platform Segmentation.

Scenario: Online streaming platform, - wants to group users based on viewing habits, like genreres, time of day, binge freq. etc. - No predefined labels or categories.

This scenario is a typical unsupervised learning problem, specifically a clustering problem.

The appropriate model for this is a clustering based unsupervised learning algorithm such as:

1. k-means clustering
2. Hierarchical clustering
3. DBSCAN

Why?

- There are no labels. The goal is to discover natural groups in user behaviour automatically.
- Clustering will group users with similar viewing patterns:

  First cluster: User's who binge watch at night
  Second cluster: user's watching family movies on weekend
  Third cluster: thriller etc.

So, unsupervised learning - clustering model is appropriate.

Q.4) Random Forest, Bias-variance & Better performance than single tree

- Random Forest:

It is an ensemble learning method based on bagging of decision trees. It builds many decision trees & combine their outputs (majority vote for classification, avg for regression)

- key Ideas for Random forest are - Bootstrap sampling & Feature randomness. In Bootstrap sampling each tree is trained on a random sample (with replacement) of training data. In feature randomness at each split, a random subset of features is considered.

- How Random Forest works

(1) From original dataset, create many bootstrap samples.

(2) For each bootstrap sample:
- Train a decision tree but at each split:
  - consider only a randomly selected subset of features.

(3) For prediction:
- classification: each tree votes.
  final class = majority vot
- Regression: output = Avg. of tree predictions

→ Bias - Variance Perspective.

- Single decision tree
  - typically low bias (can fit complex relationship)
  - But high variance: small changes in data → big changes in tree structure & predictions

. Tends to overfit, especially deep trees.

• Random forest.
: Each tree is high variance, low bias model
: But due to averaging many uncorrelated trees, the variance reduces significantly, while bias remains similar.
. Overall - low variance, reasonable bias
→ better generalization.

→ Why Random Forest can outperform a single tree in customer churn classification.

Scenario: predicting whether a customer will churn or not?

Single tree:
- May fit noise in training data
- decision boundary may be very complex, sensitive to outliers,
- overfits → high training accuracy, but lower test accuracy.

Random Forest:
- Uses different subsets of data & features of each tree, reducing correlation betn trees
- Majority voting smooths out individual tree errors.

- Handles non-linear relationship, interaction between features, and noisy features more robustly.
- Inherently performs a type of feature selection, less useful features will be chosen less often.

Therefore, Random forest typically:
- Increases prediction accuracy on unseen data
- is most stable & robust
- Achieves better bias & variance trade-off than a single decision tree, explaining its superior performance on customer churn prediction

Q.4 B. Ensemble learning: Bagging, Boosting, Stacking.

Ensemble learning combines multiple base models (weak/strong learners) to build a more accurate and robust overall model. It uses the principle "wisdom of the crowd" - combining diverse opinions reduces individual errors

It gives following benefits.
- Better generalization (low error)
- Reduced variance (bagging)
- Reduced bias (boosting)
- Can leverage strength of different model types. (Stacking).

→ Bagging (Bootstrap Aggregating)

It reduces variance by training models on different bootstrap samples & averaging their predictions

Steps:

1. generate several bootstrap samples from original data
2. Train base learner (e.g. full decision tree) on each sample independently.
3. For prediction:
   - classification: majority vote
   - Regression: Avg. predictions,

Example - Random forest is the classic bagging -based ensemble of decision trees including extra feature randomness.

→ Boosting

It builds model sequentially. each new model focuses on correcting errors of the previous ones. It is aims to reduce bias & improve performance with weak learners.

Steps:

1. Start with uniform weights on training samples
2. Train a weak learner (e.g. shallow tree)
3. Increase weights for misclassified samples
4. Train next learner with new weights.
5. Final prediction: weighted sum (or vote) of weak learners.

Examples:

- Adaboost.
- Gradient Boosting, XGBoost

Initially first model captures main trend, & then subsequent models fix what earlier models got wrong. Eventually, final ensemble becomes a strong learner.

→ Stacking

It combines different types of base models using a meta-learner

Steps

(1) Train various base models on same training data: eg. Logistic regression, decision tree, Svm, K-NN.
(2) collect their predictions on validation set.
(3) use the predictions as features to train meta-model e.g. another logistic regression or small neural Network.

④ For test data:
- Get base model predictions. Feed them to meta-model to get final prediction.

Advantage:
Stacking exploits different inductive biases. One model might capture linear patterns, second non-linear & third might capture local structure. Meta learner learns how to weight or trust each base model in different region of input space.